expectation under y of the cross-entropy between the distributions p_X and q_X conditioned on y, in formulas:

$$H(p_{X|Y}, q_{X|Y}) = \mathop{\mathbb{E}}_{y\sim p_Y} H(p_{X|Y=y}, q_{X|Y=y}) = \mathop{\mathbb{E}}_{y\sim p_Y} H(p(\cdot \mid y), q(\cdot \mid y)).$$

Definition L.2. The *relative mutual information* $I_{p\|q}(X;Y)$ of X and Y from q relative to p is defined by

$$I_{p\|q}(X;Y) = D(p \parallel q_X \otimes q_Y) - D(p \parallel q).$$

We have $I_{p\|q}(X;Y) = H(p_X, q_X) + H(p_Y, q_Y) - H(p, q)$, $I_{p\|p}(X;Y) = I_p(X;Y)$ reduces to standard mutual information under the distribution p and $I_{p\|q}(X;Y)$ is symmetric in X and Y but can be negative.

We have the following relative version of the decomposition $H(X) = H(X \mid Y) + I(X;Y)$.

Lemma L.3. $H(p_X, q_X) = H(p_{X|Y}, q_{X|Y}) + I_{p\|q}(X;Y)$.

Proof. We calculate

$$\begin{aligned}
H(p_X, q_X) &= -\sum_x p(x) \log q(x) \\
&= -\sum_{x,y} p(x,y) \log q(x) \\
&= -\sum_{x,y} p(x,y) \log \frac{q(x)q(y)}{p(x,y)} \frac{p(x,y)}{q(x,y)} \frac{q(x,y)}{q(y)} \\
&= D(p \parallel q_X \otimes q_Y) - D(p \parallel q) - \sum_{x,y} p(y)p(x \mid y) \log q(x \mid y) \\
&= I_{p\|q}(X;Y) + \sum_y p(y) H(p_{X|y}, q_{Y|y}) \\
&= I_{p\|q}(X;Y) + H(p_{X|Y}, q_{X|Y}).
\end{aligned}$$

□

Symmetrizing, we get the desired relative version of $H(X) + H(Y) = H(X \mid Y) + 2I(X;Y) + H(Y \mid X)$:

$$H(p_X, q_X) + H(p_Y, q_Y) = H(p_{X|Y}, q_{X|Y}) + 2I_{p\|q}(X;Y) + H(p_{Y|X}, q_{Y|X}).$$

Setting p to be the empirical distribution of the training data, the left-hand side describes the cross-entropy loss used to train 2-token prediction models. The right-hand side gives the decomposition into a *local* cross-entropy term, a mutual information term with weight two and a shifted next-token cross-entropy term. We interpret this as follows: by adding the term $H(p_Y, q_Y)$ to the loss, 2-token prediction incentivizes models to precompute features which will become useful for predicting Y in the next step and increases the weight of the relative mutual information term in the loss. What does relative mutual information actually mean? By interpreting Kullback-Leibler divergence $D(p \parallel q)$ as the average number of bits needed in addition to send data from p with a code optimized for q instead of p, we see that minimizing

$$I_{p\|q}(X;Y) = D(p \parallel q_X \otimes q_Y) - D(p \parallel q)$$

means minimizing the average number of additional bits needed to send data from p with a code optimized for q that treats X and Y as independent compared to one that does not. If this number is small, q managed to exploit the mutual information of X and Y under p.

L.3. Lookahead reinforces choice points

Training with multi-head prediction increases the importance of choice points in the loss in comparison to inconsequential decisions. To make this argument, we present a simplified model of language modelling. Consider a sequential decision task and a model M that is trained in a teacher-forced way on optimal trajectories. We distinguish *choice points* –transitions that lead to different outcomes – and *inconsequential* decisions which do not (Figure S17 (a) and (b)).

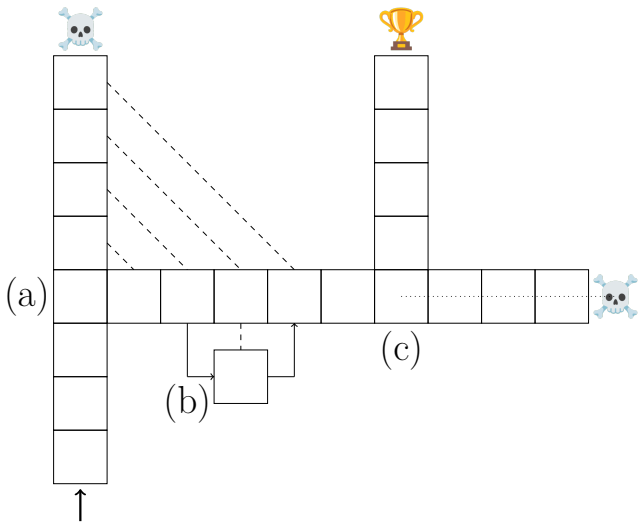


Figure S17: **Example of a sequential prediction task with derailing.** The goal is to go from the arrow to the trophy. Turning around is not allowed. Most transitions are unique, but there are two turns to be taken correctly, the *consequential decisions* (a) and (c). Turn (b) is an *inconsequential decision*: the paths join right after it. Next to transitions (a) and (b), we sketch how a 4-step prediction loss can place more emphasis on consequential transitions than inconsequential ones during teacher-forced training. Next to transition (c), we sketch how a 4-step lookahead can prevent models from taking irreversible suboptimal decisions during autoregressive decoding.

More formally, assume that the language model is deployed in a reinforcement learning setting like in *reinforcement learning from human feedback* (Ouyang et al., 2022) (states are prompts followed by the partial sequence of tokens $x_{t:1}$ generated so far, actions are single tokens x_{t+1} to generate, rewards are external $R(x_{t:1})$). The quantity

$$V_\pi(x_{t:1}) = \mathbb{E}_{x_{t+i} \sim \pi(x_{t+i-1:1}), i \geq 1} \left[\sum_{i \geq 0} R(x_{t+i:1}) \right]$$

is the value of the state $x_{t:1}$ following the policy π, while

$$\sigma_\pi(x_{t:1}) = \sqrt{\operatorname*{Var}_{x_{t+1} \sim \pi(x_{t:1})} [V_\pi(x_{t+1:1})]}$$

quantifies the importance of the decision x_{t+1} on the value thereafter. *Choice points* can formally be viewed as steps t for which $\sigma_\pi(x_{t:1})$ is large, while *inconsequential points* are steps where it is low. Note that for completion models, there is no explicit reward, and our argument is merely meant to illustrate what we mean by *choice points*.

Derailing denotes a situation where autoregressive generation of trajectories from M at inference time results in bad outcomes after M made a mistake on a choice point. Even if subsequently, M acts optimally given this choice, the final outcome can be significantly worse than the outcome of the optimal trajectory.

Staying in the teacher-forced setting, we ask: What is the impact of training M with n-step prediction instead of next-step prediction on this task? Say $x_t \to x_{t+1}$ is a choice point in an optimal trajectory with the suboptimal choice being $x_t \to \tilde{x}_{t+1}$ (Figure S17 (a)). Assume that the trajectories preceding x_t and succeeding x_{t+1} and $\tilde{x}_{t+1}$ consist of inconsequential transitions, the latter denoted by $\tilde{x}_{t+j} \to \tilde{x}_{t+j+1}$. We will compare the losses of a teacher-forced next-step prediction model and a teacher-forced n-step prediction model on the partial trajectory $(x_{t-n+1}, \ldots x_t)$. For the next-step prediction model, the predictions are $(x_{t-n+2}, \ldots, x_t, \tilde{x}_{t+1})$ with a single wrong prediction. The predictions of an n-step prediction model at time $t-n+i$, $i = 1, \ldots, n$ are $(x_{t-n+i+1}, \ldots, x_t, \tilde{x}_{t+1}, \ldots, \tilde{x}_{t+i})$ with i wrong predictions. In other words, an n-step prediction model receives $1 + \ldots + n = \frac{n(n+1)}{2}$ loss terms pertaining to such a choice point and its consequences, while each inconsequential transition (Figure S17 (b)) is only reinforced n times as often as in a next-step prediction model. In other words, choice points receive on average $\frac{n+1}{2}$ times more importance in the loss of n-step prediction models than in next-step prediction models.

As argued in Section 5.1, we believe that this model captures important features of training and inference with language models: choice points are semantically important turning points in the generated texts, such as the final answer to a question or a specific line of code, while inconsequential decisions can be a choice among synonyms or of variable names in code.

Apart from this training dynamics point of view, we hypothesize that n-step prediction also allows the formation of circuits that specifically spot inconsistencies between predictions for earlier and later steps. For instance, if in an early layer of the model, it can be predicted that a decision $x_t \to \tilde{x}_{t+1}$ leads to suboptimal outcomes $\tilde{x}_{t+n}$ (Figure S17 (c)), subsequent layers can reduce the probability of $x_t \to \tilde{x}_{t+1}$ in the model's next-step prediction. Such behaviors also happen in next-step prediction models given enough capacity, but our experiments in Section 4.2 point to the fact that circuits of this kind are formed more easily in multi-step architectures that enforce the required information $\tilde{x}_{t+n}$ to be available to the model when predicting $\tilde{x}_{t+1}$. We believe that this situation appears frequently in natural language and code modelling, for instance where an initial answer to a question contradicts the results of the *chain of thought* brought forward with the intention to justify it.

In more general terms, this situation arises whenever predicting first $\tilde{x}_{n+i}$ for some $1 < i \leq n$ and then $\tilde{x}_{n+1}$ based on $\tilde{x}_{n+i}$ is easier than predicting $\tilde{x}_{n+1}$ directly. We discuss this phenomenon of *factorization orders* in the next section and present a specific instance of it that frequently appears in modelling natural language.

L.4. Factorization orders

Causal language modelling factorizes probabilities over text sequences $x_t \cdots x_1$ classically as

$$P(x_t \cdots x_1) = \prod_{i=1}^{t} P(x_i \,|\, x_{i-1} \cdots x_1).$$

While moving forward in time is certainly the most natural choice of factorization order, there exist cases where it is suboptimal. In inflectional languages, for instance, agreement between related sentence parts is a frequent pattern with one word directing the grammatical forms of others. Consider the German sentence

> Wie konnten auch Worte meiner durstenden Seele genügen?[3]
>
> *Friedrich Hölderlin, Fragment von Hyperion (1793)*

where "genügen" requires a dative case object and then "Seele" requires the possessive pronoun "mein" to be in female singular dative form "meiner" and the participle "durstend" to be in female singular dative form in weak declination "durstenden" because it follows "meiner". In other words, the factorization order

> Wie konnten auch Worte → genügen → Seele → meiner → durstenden?

is arguably an easier one for constructing the above sentence. Humans as well as language models therefore have to perform this factorization (which deviates from the causal order in which predictions take place!) within their latent activations, and a 4-token prediction loss makes this easier as it explicitly encourages models to have all information about the successive 4 tokens in its latent representations.

[3] roughly: *How could words be enough for my thirsty soul?*

M. Training hyperparameters

Table S13: **Overview of all training hyperparameters used.** We schedule all learning rates with a linear warmup and cosine decay (Loshchilov and Hutter, 2017) to a fraction of the peak learning rate which is depicted in the last column ("decay ratio"). All experiments use the Adam (Kingma and Ba, 2015) optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$ and decoupled L_2 weight decay (Loshchilov and Hutter, 2019) coefficient 0.1. We clip gradients to a maximal Euclidean norm of 1.0 in all experiments except CodeContests finetunings, where we use 0.1 instead. Summarization finetunings correspond to three epochs on all datasets except BigPatent (1 epoch). Byte-level models use the architecture with replicated unembeddings from Appendix B.

Model	Batch size (2^{20})	Steps	Tokens (B)	Warmup steps	Peak LR	Context length	Decay ratio
Model scaling (Section 3.1)							
0.3B	8	10,850	91.0	1000	3×10^{-4}	4096	0.03
0.6B	8	10,850	91.0	1000	3×10^{-4}	4096	0.03
1.3B	8	10,850	91.0	1000	3×10^{-4}	4096	0.03
3B	8	10,850	91.0	1000	3×10^{-4}	4096	0.03
7B	8	25,000	209.7	2000	3×10^{-4}	4096	0.03
13B	8	25,000	209.7	1000	3×10^{-4}	4096	0.03
Code models (Section 3)							
7B 200B	8	25,000	209.7	2000	3×10^{-4}	4096	0.03
7B 500B	7	68,570	503.3	2000	3×10^{-4}	4096	0.03
7B 1T	7	136,240	1000.0	2000	3×10^{-4}	4096	0.03
Byte-level models (Section 3.3)							
7B 314GB	12	25,000	314.6	2000	3×10^{-4}	8192	0.03
Language models (Section 3.7)							
7B 200B	8	25,000	209.7	2000	3×10^{-4}	4096	0.10
7B 500B	8	60,000	503.3	2000	3×10^{-4}	4096	0.10
Induction task (Section 4.1)							
1M – 1B	0.25	100,000	26.2	2000	10^{-4}	2048	0.03
1M – 1B (Appendix J)	0.5	50000	26.2	2000	10^{-4}	2048	0.03
Arithmetic task (Section 4.2)							
30M	0.25	100,000	26.2	2000	10^{-4}	1024	0.03
100M	0.25	100,000	26.2	2000	10^{-4}	2048	0.03
Summarization (Section 3.7)							
BigPatent	0.125	76,680	10.1	100	3×10^{-5}	4096	0.03
CNN/Dailymail	0.125	7,140	0.9	100	3×10^{-5}	4096	0.03
Multi-News	0.125	3,330	0.4	100	3×10^{-5}	4096	0.03
OrangeSum	0.125	360	0.0	100	3×10^{-5}	4096	0.03
pn-summary	0.125	3,450	0.5	100	3×10^{-5}	4096	0.03
SAMSum	0.125	60	0.0	100	3×10^{-5}	4096	0.03
ThaiSum	0.125	23,640	3.1	100	3×10^{-5}	4096	0.03
WikiSummary	0.125	2,550	0.3	100	3×10^{-5}	4096	0.03
XSum	0.125	2,760	0.4	100	3×10^{-5}	4096	0.03
CodeContests (Section 3.6)							
7B	0.25	13,000	3.6	400	5×10^{-5}	4096	0.004